Dependency Hell

Game Treatment

High Concept

Dependency Hell is an action roguelite about surviving the software systems you built while trying to ship one more release.

At first glance, it resembles a room-based action roguelite in the direction of The Binding of Isaac: short procedural runs, strange pickups, cascading item synergies, boss encounters, escalating difficulty, and semi-permanent consequences within a run. Underneath that arcade shell, the world is a dependency graph. Every upgrade, library, tool, runtime, package, service, workaround, and "quick fix" changes the behavior of the entire system.

The player is not merely fighting enemies. The player is fighting emergent coupling.

The game should feel funny, stressful, clever, and occasionally triumphant. It should produce miniature war stories:

"I only upgraded the serializer because the auth boss was immune to legacy tokens, but that pulled in Runtime 4, which broke the logging pipeline, so I had to fight the next three rooms blind until I found a compatibility shim behind a deprecated mirror."

That is the intended magic.

Tone

The tone is darkly comic, technical, and absurd, but not purely parody. It should feel like a haunted arcade cabinet built by an SRE, a package manager, and a demon that understands semantic versioning.

The game is not edutainment. It does not stop to lecture. However, good play naturally teaches operational habits:

- isolate changes
- understand dependency graphs
- avoid unnecessary coupling
- preserve rollback paths
- test assumptions
- distrust "minor" version updates
- value observability
- never deploy unknown nonsense just because it has a nice icon

The humor comes from recognition, not from random jokes pasted over mechanics.

Core Fantasy

The player is the last engineer awake during a doomed release train, trapped inside a living software architecture. Rooms are services. Corridors are dependencies. Pickups are packages. Enemies are bugs, regressions, breaking changes, deprecated APIs, abandoned maintainers, security advisories, version conflicts, and unclear root causes.

The player progresses by choosing upgrades and fixes that make them stronger in the short term but also increase the complexity and fragility of the system.

The core emotional loop:

1. Discover an immediate problem.
2. Choose a plausible fix.
3. Gain a useful capability.
4. Trigger an unintended interaction.
5. Scramble to contain the fallout.
6. Understand what really happened.
7. Apply a clever workaround.
8. Feel competent for approximately nine seconds.
9. Repeat.

The central joke is that the player is building the maze while trying to escape it.

Genre Shape

Dependency Hell is primarily a room-based action roguelite.

The player moves through interconnected rooms, fighting enemies, dodging hazards, collecting upgrades, and making branching choices. However, instead of a simple dungeon map, the room graph is also a live dependency graph. The path between two rooms is not merely physical; it represents compatibility, imports, runtime requirements, service calls, API contracts, build steps, or deployment coupling.

The game borrows from several adjacent genres:

- From The Binding of Isaac: room-by-room combat, item synergies, weird builds, procedural runs, bosses, run-defining pickups.
- From FTL: crisis triage, cascading system failures, pausable panic, choosing which fire to fight first.
- From Cogmind: deep systems interactions, readable machine logic, procedural complexity, emergent build identity.
- From Slay the Spire: strategic route selection, tradeoff-heavy upgrades, risk evaluation, build-shaping decisions.
- From Papers, Please: bureaucratic absurdity and the slow accumulation of consequences from earlier approvals.

The result is not a software management sim. It is an arcade game where package management, incident response, and architectural entropy are expressed through combat, traversal, pickups, hazards, and map mutation.

Player Role

The player character can be represented as an engineer avatar, a tiny process, a cursor, a daemon, or a heroic terminal prompt.

Possible player identities:

- The On-Call Engineer
- The Maintainer
- The Resolver
- The Last SRE
- The Build Shepherd
- The Release Goblin
- PID 1

The avatar should be simple and readable in combat. It can have a retro vector, terminal, or low-resolution sprite aesthetic. The surrounding UI can carry most of the theme through logs, dependency overlays, alerts, status panels, and graph views.

World Structure

A run is a release cycle.

Each floor, act, or biome represents a stage in the software lifecycle:

1. Local Development
2. CI Pipeline
3. Staging
4. Pre-Production
5. Production
6. Emergency Hotfix Realm
7. Legacy Subsystem
8. Compliance Audit
9. Major Version Upgrade

Each level contains a procedural graph of rooms. The player chooses paths through services, packages, build stages, infrastructure components, test environments, and production systems.

Room types might include:

- Combat Room: fight bugs, regressions, and incompatible interfaces.
- Package Room: choose a dependency, upgrade, patch, or workaround.
- Test Room: reveal hidden incompatibilities or validate assumptions.

- Build Room: convert resources into deployable artifacts.
- Incident Room: timed triage event with cascading failures.
- Archive Room: search ancient documentation or old releases.
- Mirror Room: retrieve deprecated packages from unreliable sources.
- Vendor Room: buy enterprise-grade nonsense.
- Refactor Room: spend time/resources to reduce complexity.
- Observability Room: improve diagnostics, logs, traces, or metrics.
- Boss Room: fight a major architectural problem.
- Postmortem Room: review what went wrong and receive rewards or scars.

Core Resources

The game should avoid too many currencies, but several thematic resources can drive decisions.

Stability

The equivalent of health. When stability reaches zero, the run fails. Stability is damaged by failed builds, production incidents, broken compatibility, unhandled bugs, and cascading outages.

Velocity

A measure of how quickly the player can move, act, ship, or clear rooms. High velocity helps short-term progress but can encourage reckless decisions.

Observability

Determines how much information the player gets about enemies, rooms, dependencies, hidden conflicts, and future consequences. Low observability means more surprises.

Technical Debt

A negative resource that accumulates as the player takes shortcuts, pins versions, ignores warnings, skips tests, or accepts hacks. Technical debt may be harmless at first, but eventually causes debt events, compounding penalties, or boss mutations.

Morale

A soft fail-state or meta-resource. Morale falls when incidents drag on, tests fail repeatedly, or the player chooses brutal workarounds. Low morale can cause slower fixes, worse shop prices, or unreliable helpers.

Security Risk

A pressure meter. Ignoring CVEs, running abandoned packages, or using shady mirrors increases risk. Too much risk can trigger breach events, audit bosses, or forced upgrades.

Build Confidence

A temporary state based on tests, validation, and known-good snapshots. High build confidence makes changes safer and reveals more information. Low build confidence makes even small changes risky.

Combat

Combat should be immediate and readable, not a literal programming puzzle.

Enemies represent system failures and software hazards. They behave in ways inspired by their technical meaning.

Examples:

- Regression: looks like a previously defeated enemy but returns with altered behavior.
- Breaking Change: charges across dependency edges, damaging anything attached.
- Transitive Bug: appears harmless until it infects connected rooms.
- Version Conflict: splits into two incompatible forms that cannot both be damaged by the same weapon.
- Deprecated API: slow, obvious, but becomes poisonous if ignored too long.
- Race Condition: flickers unpredictably and only becomes vulnerable at certain timing windows.
- Memory Leak: grows slowly unless contained early.
- Circular Dependency: links two or more enemies; damaging one heals another unless the cycle is broken.
- Zombie Maintainer: occasionally wakes up, changes rules, then disappears again.
- License Violation: not immediately dangerous, but summons lawyers, auditors, or build blockers.
- Supply Chain Imposter: appears as a helpful pickup or package until installed.

The player's weapons and abilities are also system metaphors:

- Patch: basic projectile.
- Rollback: undo recent damage or revert room state.
- Feature Flag: temporarily disable part of the graph.
- Container: isolate a hazardous effect.
- Shim: translate between incompatible damage types.
- Fork: create a parallel version of a package or player effect.
- Cache: repeat the last successful action at lower cost.
- Test Suite: reveals hidden enemy properties or weakens certain foes.
- Debugger: slows time or freezes an enemy to inspect it.
- Kill -9: high damage, high collateral.
- Restart: clears some conditions but risks losing state.
- Runbook: one-time guided solution for a known incident.
- Rubber Duck: reveals a clue after the player stops moving briefly.

Dependency Graph as Game System

The dependency graph is the game's central differentiator.

Every package, service, upgrade, or workaround is represented as a node. Nodes have relationships:

- requires
- conflicts with
- replaces
- patches
- wraps
- isolates
- observes
- accelerates
- deprecates
- infects
- pins
- forks

The player sees a simplified version during normal play and can open a fuller graph view between rooms or during pauses.

Important design principle:

The graph must be legible enough to support intentional play, but incomplete enough to allow surprise.

A player should often think:

"I knew this might happen, but I hoped it wouldn't."

Not:

"That was arbitrary garbage."

Items and Upgrades

Items are packages, tools, architectural patterns, rituals, and questionable decisions.

Items should have readable benefits and hidden or delayed interactions.

Examples:

Left Pad

Small utility package.

Effect:

- Slightly increases fire rate.
- Adds one transitive dependency.

Hidden risk:

- Small chance to be unpublished during later floors.

OpenSSL

Heavy cryptographic dependency.

Effect:

- Enables secure damage type.
- Required for many advanced items.

Risk:

- High CVE event likelihood.
- Version conflicts with legacy auth.

LegacyAuth

Old authentication layer.

Effect:

- Grants access to locked rooms.
- Reduces enemy aggro from older systems.

Risk:

- Conflicts with modern crypto.
- Increases audit pressure.

Containerization

Effect:

- Reduces conflict damage between adjacent nodes.
- Prevents some transitive infections.

Risk:

- Increases startup delay.
- Adds orchestration dependency.

Kubernetes

Effect:

- Greatly increases scalability and room reward potential.
- Allows self-healing behavior.

Risk:

- Adds several new failure modes.
- Spawns YAML hazards.
- Increases observability requirements.

Enterprise Service Bus

Effect:

- Connects distant nodes.
- Converts some incompatibilities into latency instead of failure.

Risk:

- Adds global coupling.
- Bosses can attack through it.

Vendored Dependency

Effect:

- Prevents upstream changes.
- Immunity to unpublish events.

Risk:

- No automatic security patches.
- Increases technical debt over time.

Pin Version

Effect:

- Freezes a dependency and prevents surprise upgrades.

Risk:

- Accumulates upgrade pressure.
- Later forced upgrades become more dangerous.

SemVer Trust

Effect:

- Minor updates become cheaper and faster.

Risk:

- Some packages lie.

Accurate Documentation

Effect:

- Reveals hidden item properties.

Rarity:

- Legendary.

Friday Deploy

Effect:

- Immediate huge reward.

Risk:

- Everything becomes funnier and worse.

Item Synergies

Synergies should be the heart of the game.

Some synergies are beneficial:

- Containerization + LegacyAuth: isolates an old subsystem safely.
- Feature Flags + CI Pipeline: allows risky upgrades with controlled blast radius.
- Observability + Debugger: reveals enemy intent before attacks.
- Vendored Dependency + Security Team: allows manual patching without upstream risk.
- Test Suite + Rollback: failed experiments become survivable.

Some synergies are dangerous:

- Microservices + Low Observability: spawns invisible failures.
- Kubernetes + YAML Fatigue: config hazards multiply.
- LegacyAuth + Modern Crypto: authentication rooms become unstable.
- High Velocity + Low Build Confidence: every room has a chance to introduce a regression.
- Enterprise Service Bus + Circular Dependency: creates a global outage event.

Some should be absurdly rare and delightful:

- Rubber Duck + Accurate Documentation: instantly identifies root cause once per floor.
- Original Maintainer + Ancient Comment: unlocks a hidden fix.
- Perl Script + Cron: does something inexplicable but useful every 60 seconds.
- COBOL Adapter + Mainframe Room: turns a terrifying legacy zone into a shop.

Procedural Run Arc

A run begins with a relatively clean system and a simple goal:

Ship Release 1.0.

The early game presents small, understandable choices:

- add logging
- upgrade runtime
- add auth
- install database driver
- enable tests
- skip tests for speed

Each choice adds power and coupling.

Mid-game introduces external pressure:

- critical CVE
- customer feature promise
- scaling event
- maintainer abandonment
- dependency unpublish
- compliance requirement
- surprise executive demo

Late-game tests the player's accumulated architecture. The final act should feel like a reckoning. The player is not fighting a random final boss. They are fighting the consequences of their build.

Possible final bosses:

- The Major Version Upgrade
- Production
- The Monorepo
- The Distributed Monolith
- The Compliance Audit
- The Friday Release
- The Dependency Resolver
- The Unknown Unknown
- The System Nobody Understands

The final boss changes depending on the player's decisions.

For example:

- Too much technical debt summons The Rewrite.
- Too much security risk summons The Breach.
- Too much coupling summons The Distributed Monolith.
- Too many pinned packages summons The Major Version Upgrade.
- Too little observability summons The Blind Incident.

Failure and Postmortem

When the player loses, the game should not merely say "game over."

It should produce a postmortem.

Example:

POSTMORTEM

Incident: Production outage during release candidate deployment.

Root Cause: LegacyAuth required crypto-core < 3.0, but payment-service pulled crypto-core 4.2 through invoice-pdf-renderer.

Contributing Factors:

- Tests skipped in Staging.
- Observability below recommended threshold.
- Two critical dependencies pinned since Act 1.
- Friday Deploy accepted.
- Rollback path blocked by schema migration.

What Went Well:

- Cache layer isolated customer reads for 42 seconds.
- Feature flag prevented total auth collapse.
- Rubber Duck correctly identified token serialization mismatch.

Action Items:

- Add integration tests.
- Reduce dependency fanout.
- Stop trusting minor versions from GoblinSoft packages.

The postmortem should be funny, specific, and generated from actual run state. This turns failure into narrative reward.

Victory

Winning should also produce a release report.

Example:

RELEASE COMPLETE

Version: 1.0.0

Customer Impact: Minimal

Known Issues: Several, but legally acceptable

Security Risk: Medium

Technical Debt: Haunted

Morale: Unusually high

Final Architecture: A terrifying but functional braid of containers, feature flags, pinned crypto, and one shell script named do-not-delete.sh.

Victory should not imply the system is beautiful. Victory means the release shipped and the pager is quiet.

Visual Direction

The game should not look like a modern corporate dashboard. It should look like an arcade cabinet possessed by a build server.

Possible visual ingredients:

- low-resolution pixel art
- vector graphics
- CRT scanlines
- phosphor glow
- terminal overlays
- dependency graph minimap
- scrolling logs
- warning banners
- CI status lights
- oscilloscope-like traces
- package icons as collectible talismans
- occasional fake ANSI screens

The player-facing action view should stay clean enough for combat. The technical flavor can live in overlays, room names, item descriptions, postmortems, boss intros, and graph views.

The dependency graph should be visually satisfying. Nodes should pulse, edges should flicker, conflicts should arc like electrical faults, and transitive failures should spread visibly through the graph.

Audio Direction

The audio should combine retro arcade energy with office panic and machine-room dread.

Possible elements:

- arcade bleeps
- modem squeals
- hard drive chatter
- alert tones
- pager buzzes
- mechanical keyboard bursts
- distant server fan drones
- distorted startup sounds
- degraded hold music
- build-success chimes
- ominous CI failure klaxons

Music can shift based on system state:

- high stability: tight synthetic pulse
- rising technical debt: detuned layers
- incident mode: faster percussion, alert motifs
- low observability: muffled mix
- final boss: corrupted release fanfare

Humor Style

The humor should be dry, specific, and systemic.

Avoid relying only on meme references. The game should still be funny years later.

Good humor sources:

- item names
- room names
- generated postmortems
- enemy behavior
- contradictory tooltips
- management requests
- fake changelogs

- impossible release notes
- passive-aggressive documentation

Examples:

Room names:

- Staging, Allegedly
- The One Server
- Auth Again
- The Blessed Cache
- Deprecated Catacombs
- Jenkins Memorial Wing
- The Vendor Portal
- YAML Swamp
- The Transitive Depths

Pickup descriptions:

- "A small package with no dependencies. Surely harmless."
- "Marked stable by someone with confidence."
- "Works on the maintainer's machine."
- "Adds observability. Also reveals how bad things are."
- "This package is widely used, which is not the same as good."

Accessibility and Audience

The game should be playable by people who do not know software dependency management, but it should reward those who do.

For general players:

- Use color, icons, and simple terms like "requires", "conflicts", "risk", and "stability."
- Teach through interaction.
- Keep action readable and satisfying.

For technical players:

- Add deeper tooltips.
- Include authentic failure modes.
- Allow graph inspection.
- Let builds become beautifully cursed.
- Include references that feel earned.

The game should not require understanding npm, Maven, pip, apt, Docker, Kubernetes, or CI/CD systems, but players who do understand them should feel seen, possibly attacked.

MVP Shape

A strong prototype could be modest.

Minimum viable version:

- One playable character.
- One floor.
- 12 to 20 room types.
- 20 to 30 items/packages.
- 8 to 12 enemy types.
- 2 bosses.
- A simple dependency graph.
- Stability, observability, and technical debt as the main resources.
- Generated postmortem on failure.
- Generated release report on success.

The prototype's goal is to prove that item choices and dependency graph mutations create interesting, legible, funny consequences during arcade play.

Essential MVP mechanics:

1. Player clears rooms in real time.
2. Player chooses packages/upgrades.
3. Each package modifies player abilities and dependency graph state.
4. Some packages have conflicts or requirements.
5. Conflicts create combat or map consequences.
6. Technical debt accumulates from shortcuts.
7. Observability determines how much the player can see.
8. Boss adapts to the player's accumulated system shape.
9. Death generates a useful/funny postmortem.

Do not begin with a huge simulation. Begin with a toy dependency model that produces good stories.

Design Pillars

1. Every Power Has a Shadow

No upgrade should be purely good for every build. Even wonderful tools add complexity, coupling, cost, or hidden risk.

2. The Graph Is the Monster

Enemies matter, but the real antagonist is the architecture forming around the player.

3. Failure Should Be Funny and Informative

A lost run should leave the player laughing and saying, "I know exactly where I went wrong."

4. The Player Builds Their Own Doom

The best failures should be traceable to earlier decisions that seemed reasonable at the time.

5. Real Enough to Sting, Abstract Enough to Play

The game should evoke real operational experience without becoming a literal package-manager simulator.

6. Competence Is the Reward

The best moment is not merely getting a rare item. It is understanding the system well enough to save it.

Example Run Narrative

The player begins in Local Development with a basic Patch projectile and a tiny dependency graph:

app -> runtime -> standard-lib

They clear a few rooms and choose Logging Framework.

Logging reveals enemy health bars and room hazards. Good choice.

Next, the player chooses LegacyAuth because it unlocks several side rooms. It works well but adds an old crypto dependency.

Later, the player finds Modern Crypto. It offers strong damage and security benefits. The graph warns of a possible conflict, but the exact issue is hidden because observability is still low.

The player installs it.

Everything seems fine for two rooms.

Then auth enemies become unstable. Some rooms spawn Token Mismatch hazards. The player's rollback ability is disabled because a schema migration was accepted earlier.

The player finds a Containerization pickup and uses it to isolate LegacyAuth. This stabilizes auth but increases startup delay. Enemy waves now begin before all abilities are ready.

A CVE event fires. The old crypto library is vulnerable. The player must choose between patching LegacyAuth, replacing it, or accepting temporary security risk.

They patch it with a Shim. The Shim works, but adds technical debt.

By the time they reach the boss, their graph is ugly but functional.

The boss is The Major Version Upgrade.

During the fight, the boss attacks the weakest edges in the dependency graph. The player's earlier Logging Framework and Containerization choices prevent total collapse. The Shim breaks halfway through the fight. The player survives by using Feature Flag to disable a damaged subsystem and finishes the boss with Modern Crypto.

Victory report:

Release shipped. Auth is weird. Security is acceptable. Technical debt is glowing. Pager is quiet.

The player immediately wants another run.

Long-Term Expansion Ideas

- Multiple ecosystems: npm-like chaos, enterprise Java labyrinth, Python environment swamp, Linux distro package fortress, container orchestration nightmare.
- Different player classes: SRE, Staff Engineer, Security Goblin, Release Manager, Ancient Unix Wizard, Frontend Alchemist.
- Daily challenge: same dependency graph seed for all players.
- War story export: share generated postmortems.
- Custom package editor.
- Mod support for fake ecosystems.
- Terminal-only visual mode.
- Boss rush: seven production incidents in a row.
- Endless mode: keep the system alive forever.
- "Consultant Mode": fix a system someone else ruined.
- "Startup Mode": high velocity, low tests, lots of confidence.
- "Enterprise Mode": many approvals, lots of legacy systems, nobody knows who owns anything.
- "Weekend Mode": all incidents are worse and shops are closed.

Summary

Dependency Hell is an action roguelite where software architecture becomes dungeon ecology. The player fights bugs and failures in real time while choosing packages, upgrades, patches, and shortcuts that mutate a live dependency graph. Every decision creates power and risk. Every run becomes a story about hidden coupling, delayed consequences, and the joy of understanding a broken system just in time to save it.

It is funny because it is true.

It is stressful because it is familiar.

It is satisfying because dependency hell, like a good roguelite run, is not solved by having the perfect plan. It is survived by making good decisions with incomplete information, adapting quickly, and learning to love the cursed machine long enough to ship.